Презентация по лабораторной работе №4

Линейная алгебра

Компьютерный практикум по стат. анализу данных

2026 г.

Российский университет дружбы народов, Москва, Россия

Информация

Докладчик

- Танрибергенов Эльдар
- студент 4 курса из группы НПИбд-01-22
- ФМиЕН, кафедра прикладной информатики и теории вероятностей
- Российский университет дружбы народов

Цели и задачи

Цель работы

Основной целью работы является изучение возможностей специализированных пакетов Julia для выполнения и оценки эффективности операций над объектами линейной алгебры.

Задачи

- Выполнить:
  - Поэлементные операции над многомерными массивами
  - Транспонирование, след, ранг, определитель и инверсия матрицы
  - Вычисление нормы векторов и матриц, повороты, вращения
  - Матричное умножение, единичная матрица, скалярное произведение
- Изучить:
  - Факторизацию. Специальные матричные структуры
  - Общую линейная алгебра

Результаты

Поэлементные операции над многомерными массивами

```
[10]: # Массив 4x3 со случайными целыми числами (от 1 до 20):
a = rand(1:20,(4,3))
# Поэлементная сумма:
println("Поэлементная сумма = ", sum(a))
# Поэлементная сумма по столбцам:
println("Поэлементная сумма по столбцам = ", sum(a,dims=1))
# Поэлементная сумма по строкам:
println("Поэлементная сумма по строкам = ", sum(a,dims=2))
# Поэлементное произведение:
println("Поэлементное произведение = ", prod(a))
# Поэлементное произведение по столбцам:
println("Поэлементное произведение по столбцам = ", prod(a,dims=1))
# Поэлементное произведение по строкам:
println("Поэлементное произведение по строкам = ", prod(a,dims=2))
```

```
Поэлементная сумма = 129
Поэлементная сумма по столбцам = [41 47 41]
Поэлементная сумма по строкам = [33; 31; 44; 21;;]
Поэлементное произведение = 227412541440
Поэлементное произведение по столбцам = [6048 6528 5760]
Поэлементное произведение по строкам = [648; 680; 3072; 168;;]
```

Рис. 1: Поэлементные операции сложения и произведения

```
[11]: # Подключение пакета Statistics:
import Pkg
Pkg.add("Statistics")
using Statistics
# Вычисление среднего значения массива:
println("Среднее значение массива = ", mean(a))
# Среднее по столбцам:
println("Среднее по столбцам = ", mean(a,dims=1))
# Среднее по строкам:
println("Среднее по строкам = ", mean(a,dims=2))
```

```
   Resolving package versions...
     Project No packages added to or removed from `C:\Users\eldar\.julia\environments\v1.12\Project.toml`
    Manifest No packages added to or removed from `C:\Users\eldar\.julia\environments\v1.12\Manifest.toml`
Среднее значение массива = 10.75
Среднее по столбцам = [10.25 11.75 10.25]
Среднее по строкам = [11.0; 10.333333333333334; 14.666666666666666; 7.0;;]
```

Рис. 2: Средние значения

Транспонирование, след, ранг, определитель и инверсия матрицы

```
[12]: # Подключение пакета LinearAlgebra:
      import Pkg
      Pkg.add("LinearAlgebra")
      using LinearAlgebra
      # Массив 4x4 со случайными целыми числами (от 1 до 20):
      b = rand(1:20,(4,4))
      # Транспонирование:
      transpose(b)

        Resolving package versions...
          Project No packages added to or removed from `C:\Users\alder\.julia\environments\v1.12\Project.toml`
          Manifest No packages added to or removed from `C:\Users\alder\.julia\environments\v1.12\Manifest.toml`
[12]: 4×4 transpose(::Matrix{Int64}) with eltype Int64:
        6  10  18   5
       11  18  11  11
       18   2  12   7
       17  10  11  16
```

Рис. 3: Создание массива и его транспонирование

```
[13]: # След матрицы (сумма диагональных элементов):
      tr(b)

[13]: 52
```

Рис. 4: След матрицы

```
[14]: # Извлечение диагональных элементов как массив:
      diag(b)

[14]: 4-element Vector{Int64}:
        6
       18
       12
       16
```

Рис. 5: Извлечение диагональных элементов как массив

Транспонирование, след, ранг, определитель и инверсия матрицы

```
[15]: # Ранг матрицы:
      rank(b)

[15]: 4
```

Рис. 6: Ранг матрицы

```
[16]: # Инверсия матрицы (определение обратной матрицы):
      inv(b)

[16]: 4×4 Matrix{Float64}:
       -0.0656058  -0.0307091    0.0862647    0.0295924
        0.0625349   0.110832    -0.0552764   -0.0977108
        0.103015    0.00921273  -0.00921273  -0.108878
       -0.06756    -0.0706309    0.0150754    0.168063
```

Рис. 7: Инверсия матрицы

```
[17]: # Определитель матрицы:
      det(b)

[17]: -21492.000000000004
```

Рис. 8: Определитель матрицы

```
[18]: # Псевдобратная функция для прямоугольных матриц:
      pinv(b)

[18]: 4×4 Matrix{Float64}:
       -0.0656058  -0.0307091    0.0862647    0.0295924
        0.0625349   0.110832    -0.0552764   -0.0977108
        0.103015    0.00921273  -0.00921273  -0.108878
       -0.06756    -0.0706309    0.0150754    0.168063
```

Рис. 9: Псевдобратная функция для прямоугольных матриц

Вычисление нормы векторов и матриц, повороты, вращения

```
[19]: # Создание вектора X:
      X = [2, 4, -5]
      # Вычисление евклидовой нормы:
      println("Евклидова норма = ", norm(X))
      # Вычисление p-нормы:
      p = 1
      println("p-норма = ", norm(X,p))
      # Расстояние между двумя векторами X и Y:
      X = [2, 4, -5];
      Y = [1,-1,3];
      println("Расстояние между двумя векторами = ", norm(X-Y))

      Евклидова норма = 6.708203932499369
      p-норма = 11.0
      Расстояние между двумя векторами = 9.486832980505138
```

Рис. 10: Евклидова и p- нормы; Евклидово расстояние

```
[20]: # Проверка по базовому определению:
      println("Проверка по базовому определению = ", sqrt(sum((X-Y).^2)))
      # Угол между двумя векторами:
      println("Угол между двумя векторами = ", acos((transpose(X)*Y)/(norm(X)*norm(Y))))

      Проверка по базовому определению = 9.486832980505138
      Угол между двумя векторами = 2.4404307889469252
```

Рис. 11: Проверка по баз. опред. и угол между векторами

Вычисление нормы векторов и матриц, повороты, вращения

Вычисление нормы для двумерной матрицы

```
[21]: # Создание матрицы:
d = [5 -4 2 ; -1 2 3; -2 1 0]
# Вычисление Евклидовой нормы:
println("Евклидова норма = ", opnorm(d))
# Вычисление p-нормы:
p=1
println("p-норма = ", opnorm(d,p))

Евклидова норма = 7.147682841795258
p-норма = 8.0
```

Рис. 12: Евклидова и и p- нормы для 2мерной матрицы

```
[23]: # Поворот на 180 градусов:
rot180(d)

[23]: 3×3 Matrix{Int64}:
 0   1  -2
 3   2  -1
 2  -4   5
```

Рис. 13: Поворот на 180 градусов

```
[25]: # Переворачивание строк:
reverse(d,dims=1)

[25]: 3×3 Matrix{Int64}:
 -2   1  0
 -1   2  3
  5  -4  2
```

Рис. 14: Переворачивание строк

```
[26]: # Переворачивание столбцов
reverse(d,dims=2)

[26]: 3×3 Matrix{Int64}:
 2  -4   5
 3   2  -1
 0   1  -2
```

Рис. 15: Переворачивание столбцов

Матричное умножение, единичная матрица, скалярное произведение

```
[27]: # Матрица 2x3 со случайными целыми значениями от 1 до 10:
      A = rand(1:10,(2,3))

[27]: 2×3 Matrix{Int64}:
       2  8  4
       2  2  7

[28]: # Матрица 3x4 со случайными целыми значениями от 1 до 10:
      B = rand(1:10,(3,4))

[28]: 3×4 Matrix{Int64}:
       4  9  3  6
       1  1  4  8
       8  3  9  10
```

Рис. 16: Матрицы со случ. цел. значениями

```
[29]: # Произведение матриц A и B:
      A*B

[29]: 2×4 Matrix{Int64}:
       48  38  74  116
       66  41  77   98
```

Рис. 17: Произведение матриц

```
[30]: # Единичная матрица 3x3:
      Matrix{Int}(I, 3, 3)

[30]: 3×3 Matrix{Int64}:
       1  0  0
       0  1  0
       0  0  1
```

Рис. 18: Единичная матрица

```
[31]: # Скалярное произведение векторов X и Y:
      X = [2, 4, -5]
      Y = [1,-1,3]
      dot(X,Y)

[31]: -17

[32]: # тоже скалярное произведение:
      X'Y

[32]: -17
```

Рис. 19: Скалярное произведение векторов X и Y

Факторизация. Специальные матричные структуры

- Julia позволяет вычислять LU-факторизацию и определяет составной тип факторизации для его хранения
- Различные части факторизации могут быть извлечены путём доступа к их специальным свойствам

```
[37]: # LU-факторизация:
      Alu = lu(A)
```

```
[37]: LU{Float64, Matrix{Float64}, Vector{Int64}}
      L factor:
      3×3 Matrix{Float64}:
       1.0         0.0      0.0
       0.256084    1.0      0.0
       0.0601857  -0.27905  1.0
      U factor:
      3×3 Matrix{Float64}:
       0.913299   0.934554   0.627835
       0.0       -0.0712758  0.492419
       0.0        0.0        0.132446
```

Рис. 20: LU-факторизация

Факторизация. Специальные матричные структуры

Свойства

```
[38]: # Матрица перестановок:
      Alu.P

[38]: 3×3 Matrix{Float64}:
       0.0  0.0  1.0
       0.0  1.0  0.0
       1.0  0.0  0.0
```

Рис. 21: Матрица перестановок

```
[39]: # Вектор перестановок:
      Alu.p

[39]: 3-element Vector{Int64}:
       3
       2
       1
```

Рис. 22: Вектор перестановок

```
[40]: # Матрица L:
      Alu.L

[40]: 3×3 Matrix{Float64}:
       1.0         0.0      0.0
       0.256084    1.0      0.0
       0.0601857  -0.27905  1.0
```

Рис. 23: Матрица L

```
[41]: # Матрица U:
      Alu.U

[41]: 3×3 Matrix{Float64}:
       0.913299   0.934554   0.627835
       0.0       -0.0712758  0.492419
       0.0        0.0        0.132446
```

Рис. 24: Матрица U

Факторизация. Специальные матричные структуры

Решение СЛАУ

```
[33]: # Задаём квадратную матрицу 3x3 со случайными значениями:
      A = rand(3, 3)

[33]: 3×3 Matrix{Float64}:
       0.0549676  0.0761364  0.0328232
       0.233882   0.168049   0.653198
       0.913299   0.934554   0.627835
```

Рис. 25: Квадратная матрица 3x3

```
[35]: # Задаём вектор b:
      b = A*x

[35]: 3-element Vector{Float64}:
       0.1639271578901671
       1.0551286182079866
       2.475688944187798
```

Рис. 26: Вектор b

```
[43]: # Решение СЛАУ через объект факторизации:
      Alu\b

[43]: 3-element Vector{Float64}:
       1.000000000000001
       0.9999999999999988
       1.0
```

Рис. 27: Решение через объект факторизации

Факторизация. Специальные матричные структуры

- Julia позволяет вычислять QR-факторизацию и определяет составной тип факторизации для его хранения
- По аналогии с LU-факторизацией различные части QR-факторизации могут быть извлечены путём доступа к их специальным свойствам

[46]:
```
# QR-факторизация:
Aqr = qr(A)
```

[46]:
```
LinearAlgebra.QRCompactWY{Float64, Matrix{Float64}, Matrix{Float64}}
Q factor: 3×3 LinearAlgebra.QRCompactWYQ{Float64, Matrix{Float64}, Matrix{Float64}}
R factor:
3×3 Matrix{Float64}:
 -0.944371  -0.949855   -0.770859
  0.0        0.0721371  -0.460089
  0.0        0.0        -0.126559
```


Рис. 28: QR-факторизация

Факторизация. Специальные матричные структуры

Свойства

```
[47]: # Матрица Q:
      Aqr.Q

[47]: 3×3 LinearAlgebra.QRCompactWYQ{Float64, Matrix{Float64}, Matrix{Float64}}
```

Рис. 29: Матрица Q

```
[48]: # Матрица R:
      Aqr.R

[48]: 3×3 Matrix{Float64}:
       -0.944371  -0.949855   -0.770859
        0.0        0.0721371  -0.460089
        0.0        0.0        -0.126559
```

Рис. 30: Матрица R

```
[49]: # Проверка, что матрица Q - ортогональная:
      Aqr.Q'*Aqr.Q

[49]: 3×3 Matrix{Float64}:
        1.0          5.55112e-17  -1.11022e-16
        5.55112e-17  1.0          -5.55112e-17
       -2.22045e-16  5.55112e-17   1.0
```

Рис. 31: Проверка, что матрица Q - ортогональная

Общая линейная алгебра

```
[67]: # Матрица с рациональными элементами:
      Arational = Matrix{Rational{BigInt}}(rand(1:10, 3, 3))/10

[67]: 3×3 Matrix{Rational{BigInt}}:
        1    1//10  3//10
      1//2   1//5   3//10
      2//5   9//10  4//5
```

Рис. 32: Матрица с рациональными элементами

```
[69]: # Задаём вектор b:
      b = Arational*x

[69]: 3-element Vector{Rational{BigInt}}:
        7//5
          1
        21//10
```

Рис. 33: Вектор b

```
[70]: # Решение исходного уравнения получаем с помощью функции \
      # (убеждаемся, что x - единичный вектор):
      Arational\b

[70]: 3-element Vector{Rational{BigInt}}:
        1
        1
        1
```

Рис. 34: Решение с помощью функции \

```
[72]: # LU-разложение:
      lu(Arational)

[72]: LU{Rational{BigInt}, Matrix{Rational{BigInt}}, Vector{Int64}}
      L factor:
      3×3 Matrix{Rational{BigInt}}:
        1      0      0
      2//5     1      0
      1//2   15//86   1
      U factor:
      3×3 Matrix{Rational{BigInt}}:
        1    1//10   3//10
        0   43//50   17//25
        0     0      27//860
```

Рис. 35: LU-разложение

Выводы

Вывод

В результате выполнения лабораторной работы, я изучил возможности специализированных пакетов Julia для выполнения и оценки эффективности операций над объектами линейной алгебры.